Blatt 04: Stream-API & Damaged Bridge (Git Remote, Streams, Lambda-Ausdrücke)

Carsten Gips (HSBI)

Unterlagen

- PDF-Version

---

1 Zusammenfassung

Auf diesem Blatt üben Sie den Umgang mit Git Remotes sowie die Erstellung von Pull-Requests auf GitHub. Darüber hinaus üben wir den Umgang mit der Java-Stream-API und noch einmal mit Lambda-Ausdrücken.

Hinweis: Bitte denken Sie daran, dass Sie spätestens ab diesem Blatt die Bearbeitung der Aufgaben in Ihren öffentlich sichtbaren Git-Repos durchführen sollen.
Erstellen Sie ab diesem Blatt für **alle** Ihre Lösungen passende Pull-Requests gegen **Ihre eigenen Repos** (auch wenn dies nicht explizit in den Aufgaben gefordert wird).
Die Links zu Ihren Pull-Requests mit den Lösungen geben Sie bitte ab sofort immer in Ihrem *Post Mortem* mit an.

2 Aufgaben

2.1 Stream-API

2.1.1 Git: Pull-Requests (und Code-Formatierung und -Dokumentation)

Forken Sie das "Stream-API"-Repo und erzeugen Sie eine lokale Arbeitskopie von Ihrem Fork.

Sie finden die Vorgaben für jede Teilaufgabe in einem eigenen Branch. Checken Sie diese Branches lokal als Tracking-Branches aus und bearbeiten Sie die Teilaufgaben jeweils in ihrem eigenen Branch. Pushen Sie Ihre Änderungen in Ihren Fork zurück und erstellen Sie dort je einen Pull-Request auf **Ihren** eigenen `master`-Branch.

Bitte lassen Sie die Pull-Requests bis zur Vorstellung im Praktikum offen.

Achten Sie darauf, alle Schritte nachvollziehbar in Ihrer Arbeitskopie per Git-Commit festzuhalten. Demonstrieren Sie im Praktikum, wie Sie mit den Pull-Requests arbeiten.

2.1.2 Stream-API: Task I

Betrachten Sie den Branch `task_i`. Sie finden im Package `streamapi` einige Hilfsklassen sowie in der Datei `Main.java` einen Starter für diese erste Teilaufgabe.

In der Funktion `Main#students` wird für eine Liste von `Student`-Objekten die Summe der gesammelten ECTS berechnet.

Schreiben Sie den Body dieser Methode so um, dass die selbe Funktionalität unter Nutzung der Java-Stream-API erreicht wird. Bevorzugen Sie dabei nach Möglichkeit Methoden-Referenzen vor Lambda-Ausdrücken.

2.1.3 Stream-API: Task II

Betrachten Sie nun den Branch `task_ii`. Sie finden wieder im Package `streamapi` einige Hilfsklassen sowie in der Datei `Main.java` einen Starter für diese zweite Teilaufgabe.

In der Funktion `Main#ifmCps` wird für eine Liste von `Student`-Objekten die Menge der gesammelten ECTS bestimmt. Dabei werden nur Studierende des Informatik-Studiengangs berücksichtigt.

Was bedeutet "die Menge der gesammelten ECTS", was ist die Mengen-Eigenschaft?

Schreiben Sie den Body dieser Methode so um, dass die selbe Funktionalität unter Nutzung der Java-Stream-API erreicht wird. Bevorzugen Sie dabei nach Möglichkeit Methoden-Referenzen vor Lambda-Ausdrücken.

2.1.4 Stream-API: Task III

Betrachten Sie nun den Branch `task_iii`. Sie finden wieder im Package `streamapi` einige Hilfsklassen sowie in der Datei `Main.java` einen Starter für diese dritte Teilaufgabe.

In der Funktion `Main#random` werden zunächst zehn zufällige Integerwerte im Bereich zwischen 0 (inklusive) und 10 (exklusive) berechnet. Anschließend werden diese Zahlen weiter verarbeitet und das Ergebnis zurückgeliefert.

Schreiben Sie den Body dieser Methode so um, dass die selbe Funktionalität unter Nutzung der Java-Stream-API erreicht wird. Bevorzugen Sie dabei nach Möglichkeit Methoden-Referenzen vor Lambda-Ausdrücken.

2.1.5 Stream-API: Task IV+V

Betrachten Sie nun den Branch `task_iv_v`. Sie finden wieder im Package `streamapi` einige Hilfsklassen sowie in der Datei `Main.java` einen Starter für diese vierte Teilaufgabe.

1. Ressourcen in Java

   In der Funktion `Main#getResourceAsStream` soll eine Textdatei als `InputStream` zum Einlesen geöffnet werden. Die Datei soll dabei im Ressourcen-Ordner des Projekts gesucht werden.

   Informieren Sie sich im Web über den Umgang in Java mit Ressourcen. Welcher Ordner wurde in der vorgegebenen Gradle-Konfiguration als Ressourcen-Ordner für das Projekt definiert?

   Schreiben Sie den Body dieser Methode so um, dass die zu dem übergebenen Dateinamen passende Ressource im Kontext der aktuellen Klasse als `InputStream` geöffnet wird und geben Sie diesen als Ergebnis zurück.

   Hinweis: Diese Teilaufgabe ist die Voraussetzung für die Bearbeitung der nächsten Teilaufgabe. Wenn Sie diese Aufgabe nicht hinbekommen sollten, dann definieren Sie sich ersatzweise einen *Text Block* (Multi-line String) mit dem Inhalt der Textdatei und erzeugen daraus einen `InputStream`. Das zählt dann aber nicht als "bearbeitet".

2. Einlesen von Textdateien

   In der Funktion `Main#resources` wird eine Textdatei im Ressourcen-Ordner über eine interne Hilfsfunktion (siehe vorige Teilaufgabe) als `InputStream` zum Einlesen geöffnet. Über einen `BufferedReader` werden alle Zeilen eingelesen und anschließend alle Zeilen, die mit dem Buchstaben "a" beginnen und mindestens zwei Zeichen lang sind, wieder zusammen gefügt (mit einem Zeilenumbruch `\n` als Trenner).

   Schreiben Sie den Body dieser Methode so um, dass die selbe Funktionalität unter Nutzung der Java-Stream-API erreicht wird. Bevorzugen Sie dabei nach Möglichkeit Methoden-Referenzen vor Lambda-Ausdrücken.

2.2 Record-Klassen

Machen Sie aus der Klasse `streamapi.Student` eine Record-Klasse.

2.3 DevDungeon: Zerbrechende Tiles und Speed Potions (Lambda-Ausdrücke)

Klonen Sie das Projekt DevDungeon und laden Sie es in Ihrer IDE als Gradle-Projekt. Betrachten Sie das Sub-Projekt[1] "devDungeon". Dies ist ein von einem Studierenden (@Flamtky) erstelltes Spiel mit mehreren Leveln, in denen Sie spielerisch verschiedene Aufgaben *in-game* und *ex-game* lösen müssen.

Starten Sie den DevDungeon mit `./gradlew devDungeon:runDevDungeon`. Spielen Sie sich für diese Aufgabe durch das **erste Level** ("Damaged Bridge")[2].

Ziel ist es, die mysteriöse Brücke in der Mitte des ersten Levels lebendig zu überqueren. Beobachten Sie die Startsequenz: Was fällt ihnen an dem Monster auf, dass Sie ganz am Anfang angreifen will? Sie finden weitere Hinweise in den Briefkästen und über die Popups … Beachten Sie auch die Hinweise am versteckten Item. Um dieses nutzbar zu machen, müssen Sie in den Java-Code des Spiels gehen (im `src/`-Unterordner im Sub-Projekt "devDungeon") und den Effekt für das Item reparieren (implementieren). Analysieren Sie den Code für das Item und seinen Effekt, und schauen Sie sich die anderen Effekte im selben Package an. Schreiben Sie nun Code für die mit "TODO" markierte Methode des Effekts. Starten Sie dann das Spiel neu und schauen Sie, ob das Item nun funktioniert.

WICHTIG: **Bevor** Sie mit der Implementierung beginnen, schauen Sie sich bitte die Einführung in die Programmierung des Dungeons und die verwendete *Entity-Component-System*-Architektur in der Lektion Intro Dungeon an. Dort werden Ihnen Hintergründe zum Dungeon und der für diese Aufgabe relevanten Component erklärt.

Hinweis: Sie können das Demo-Level deaktivieren, indem Sie in der Klasse `starter.DevDungeon` das Flag `SKIP_TUTORIAL` auf den Wert `true` setzen. Damit gelangen Sie direkt in das in dieser Aufgabe relevante Level.

[1] Gradle-Subprojekte sind im Prinzip mehrere Java-Projekte in einem gemeinsamen Repository mit einer gemeinsamen Gradle-Basiskonfiguration. Jedes Sub-Projekt hat dann noch einmal eine eigene, die Basiskonfiguration verfeinernde Gradle-Konfiguration. Da jedes Sub-Projekt eigene Tasks mitbringen kann, muss denn der Name des Sub-Projekts dem Tasknamen vorangestellt werden: Beispielsweise muss statt `./gradlew runDevDungeon` nun `./gradlew devDungeon:runDevDungeon` aufgerufen werden. Siehe auch Multi-Project Build Basics oder Structuring Projects with Gradle.

[2] Das erste richtige Level, also das erste Level *nach* dem Demo-Level. Das Demo-Level zeigt Ihnen, wie Sie das Spiel bedienen können. Zusätzlich gibt es die kurze Anleitung "How to play" …

Hinweis: Aktuell ist das Projekt DevDungeon an einigen Stellen noch *Work-in-Progress*, beispielsweise fehlt häufig noch die Javadoc. Alle Gradle-Tasks, die von Checkstyle-Tasks abhängen (`checkstyleMain`, `check`, `build`, ...) werden deshalb fehlschlagen. Sie können den DevDungeon aber wie oben beschrieben mit `./gradlew devDungeon:runDevDungeon` (bzw. über den Task `devDungeon:runDevDungeon` aus der IDE heraus) starten.

WICHTIG: Achten Sie bitte darauf, dass im Projektpfad **keine Leerzeichen** und keine Sonderzeichen (Umlaute o.ä.) vorkommen! Dies kann zu seltsamen Fehler führen. Bitte auch darauf achten, dass Sie als JDK ein **Java SE 21 (LTS)** verwenden. Unter Windows ist der Einsatz von WSL empfehlenswert.

3 Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung oder bis zu 3er Teams
- Abgabe:
  - Post Mortem im ILIAS eintragen:

    Verfassen Sie im ILIAS pro Blatt und pro Team-Mitglied ein aussagekräftiges und nachvollziehbares "*Post Mortem*". Gehen Sie dabei auf folgende Punkte ein:

    1. Zusammenfassung: Was wurde gemacht?
    2. Implementierungsdetails: Kurze Beschreibung besonders interessanter Aspekte der Umsetzung.
    3. Was war der schwierigste Teil bei der Bearbeitung? Wie haben Sie dieses Problem gelöst?
    4. Was haben Sie gelernt oder (besser) verstanden?
    5. Team: Mit wem haben Sie zusammengearbeitet?
    6. Links zu Ihren Pull-Requests mit der Lösung.

    Das Post Mortem muss von **jeder Person** im Team **individuell** verfasst und abgegeben werden. Der Umfang des Textes soll zwischen 200 und 400 Wörtern liegen.

    Laden Sie hier bitte **nicht** Ihre Lösungen hoch!
  - Deadline: 23. Mai, 08:00 Uhr
- Vorstellung im Praktikum: 23. Mai

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 4abb344 (homework: point windows users to 'windows subsystem for linux' (wsl) for dungeon tasks, 2025-05-27)